Retrofit Lab: Sandbox App

Technical task

1. Goal

Build a small, separate Android app to practice Retrofit by hand. No old code, no old navigation, no old bugs. When it works, you can move the image search into your notes app.

Rules for this task:

- You write all the code yourself.
- Ask me for hints, not for full answers.
- After each stage, send me your code for review.

2. Tech stack

- Kotlin, Jetpack Compose, Material 3
- Retrofit 3.0.0 + converter-gson 3.0.0
- OkHttp logging-interceptor 4.12.0
- Coroutines and StateFlow
- Coil for images (same as in your notes app)
- Architecture: MVI like in RoomDbPractice (Intent, State, ViewModel)
- Navigation: none. Use a TabRow with 3 tabs and a `when`.

Setup rules (we learned these from the Room dependency problems):

- Start from the Android Studio New Project template.
- Do not change AGP, Kotlin or other versions by yourself.
- Add one dependency at a time. Sync and build after each one.
- Add the INTERNET permission to the manifest.
- Do not push API keys to GitHub. Optional: keep the key in `local.properties`.

3. The three tabs

Tab	API	Base URL	Key?	Main skills
Posts	JSONPlaceholder	https://jsonplaceholder.typicode.com/	No	@GET, @Query, @Path, @POST, @Body

Tab	API	Base URL	Key?	Main skills
Images	Pixabay	https://pixabay.com/	Yes (free)	Search, wrapper class, Interceptor
Weather	Open-Meteo	https://api.open-meteo.com/	No	Nested JSON, @SerializedName

Pexels is not used, because it paused new API keys.

Tab 1: Posts

1. Show the list of posts from `GET posts`.
2. Filter by user (userId 1 to 10) with `@Query`.
3. Tap a post to see its details with `GET posts/{id}` using `@Path`.
4. Create a post: a form with title and body, then `POST posts` with `@Body`. Show the answer. The server only pretends to save, so the new post will not appear in the list.
5. Optional: show comments with `GET posts/{id}/comments`.

Tab 2: Images

1. A search field and a Search button. No debounce yet.
2. Call the Pixabay search (the `api/` endpoint with `q` and `image_type=photo`). The answer is a wrapper object, and the list of images is inside it. Read the docs and take only the fields you need.
3. Show the results in a LazyVerticalGrid with Coil.
4. If the query is blank, do nothing.
5. Show three states: loading, error, empty result.
6. Tap an image to see it bigger and save its URL in the state. (The notes app needs exactly this later.)
7. First send the key as a `@Query` in the function. In Stage 5, move it to an Interceptor.

Tab 3: Weather

1. Call `v1/forecast` with latitude, longitude and the `current` parameter (temperature and wind speed). Check the docs for the exact names.
2. Add 3 or 4 city buttons with fixed coordinates.
3. The answer has a nested object and snake_case names. Use `@SerializedName` and Kotlin style names in your classes.
4. Show temperature and wind speed.

4. Network layer rules

- Use one network object with `by lazy`. Every base URL ends with `/`. You have three base URLs: decide how to build the Retrofit instances, and explain why.
- DTO classes (server data) are separate from the UI State. Use only the fields you need.
- All API functions are `suspend`. Choose one style: return the body, or return `Response<T>`. Use it everywhere and explain why.
- Interceptors: one for logging (you see URLs and JSON in Logcat), and one custom Interceptor that adds the Pixabay key to every request.
- Use a sealed class for results: Loading, Success(data), Error(message). The ViewModel puts it into the State.
- Handle these cases: no internet (test with airplane mode), server error or wrong key (change one letter of the key), empty result, wrong JSON (read the Logcat).

5. MVI rules (from RoomDbPractice)

- Each tab has an Intent (sealed interface), a State (data class) and a ViewModel.
- The ViewModel has a private `_state` and a public `state`. Inside the ViewModel, read only `_state.value`.
- The State is immutable. Use `copy()` to change it.
- The UI reads the state with `collectAsState` and sends Intents. It never changes the state directly.

6. Stages

Send me your code for review after each stage.

Stage	Task	Done when
0	Setup: new project, dependencies, INTERNET permission, TabRow with 3 empty tabs	The app runs
1	Posts: the list	The list shows on screen
2	Posts: filter by user and details	`@Query` and `@Path` work
3	Posts: create a post	`@POST` and `@Body` work and you see the answer
4	Images: search with the key in `@Query`	The grid shows real photos

Stage	Task	Done when
5	Interceptors: logging and key. Also call https://httpbin.org/headers with your own `@Header` and read it back	The key is not in the functions, Logcat shows the requests, the header is echoed
6	Result wrapper and error handling on all tabs	Airplane mode shows a clear message and the app does not crash
7	Weather and `@SerializedName`	Temperature shows on screen
8	Bonus, pick any: debounce while typing, pagination (`page` and `per_page`), favorite images in Room, upload a gallery image with `@Multipart` and `@Part` to a test server like httpbin.org/post	You can explain how it works

After Stage 7: move the image search into the notes app.

7. Skills map (from your checklist)

Skill	Stage
Retrofit.Builder, baseUrl, converter	1
@GET	1
@Query	2 and 4
@Path	2
@POST and @Body	3
@Header and @Headers	5
@Multipart (optional)	8
suspend functions and coroutines	all stages
try/catch and Result wrapper	6
OkHttp Interceptor (key and logging)	5
Public API + Compose screen (the practical check)	1 to 7

8. Self-check questions

Answer in your own words, without looking at notes.

1. What does `.create(Api::class.java)` do?
2. Why must the base URL end with `/`?
3. When do you use `@Path`, `@Query` and `@Body`?
4. Why does Pixabay need a wrapper class, but `GET posts` does not?
5. What happens when a JSON name is different from your Kotlin field name? How do you fix it?
6. Why is an Interceptor better than adding the key in every function?
7. What is the difference between no internet and an HTTP 404? How does your app show each one?
8. Why can `catch (e: Exception)` be a problem inside a coroutine?

9. Definition of done

- Stages 0 to 7 work.
- The app does not crash in airplane mode.
- You can answer at least 7 of the 8 questions.
- Then the Retrofit block is done. Stage 8 is a bonus.